Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 09

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

Documentación del laboratorio · Semana 9

Interfaz clínica con React y Vite

Desarrollé un frontend de una sola página para la clínica veterinaria con React y Vite. La interfaz utiliza datos de ejemplo en memoria que respetan los campos y relaciones publicados por la API de la Semana 8. En este laboratorio no se realizan solicitudes al backend; la integración con la API queda para una etapa posterior.

Ejercicio 1. Entorno Node.js y npm

Comprobé el entorno de ejecución instalado y declaré el mínimo requerido por Vite en package.json.

Comando	Resultado real
node -v	v24.19.0
npm -v	11.9.0

Correspondencia aplicada: Node.js/npm proveen el entorno y las dependencias del frontend, del mismo modo que Python/pip se usan para el backend; React compone la interfaz y Django gestiona rutas, modelos, persistencia y API.

Node/npm requirement and project commands

package.json · Extracto literal del archivo fuente

```
  "type": "module",
  "engines": {
    "node": ">=20.19.0"
  },
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
```

Imagen 1. Extracto literal de package.json: versión mínima de Node.js y scripts del frontend.

Ejercicio 2. Crear y ejecutar el proyecto Vite

Configuré el cliente en Semana_09/frontend, instalé las dependencias y ejecuté el servidor de desarrollo. Vite presentó el panel en http://localhost:5173/ y el navegador cargó la aplicación sin errores.

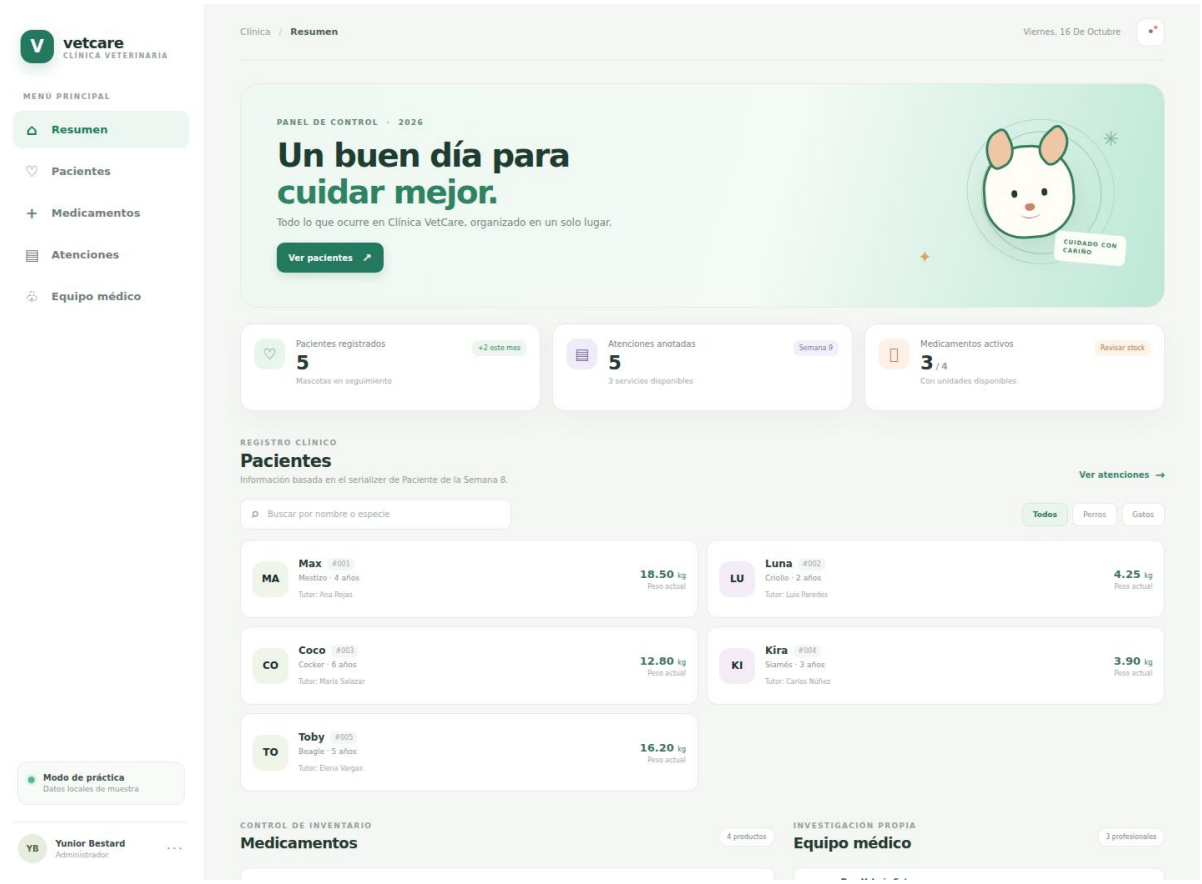


Imagen 2. Captura real del navegador con el panel inicial de VetCare.

Ejercicio 3. Estructura del proyecto

Identifiqué index.html como documento anfitrión, main.jsx como punto de entrada y App.jsx como componente principal. package.json y package-lock.json registran scripts y dependencias; node_modules y dist se excluyen del control de versiones.

Vite entry point and React mount

main.jsx · Extracto literal del archivo fuente

```
1  import { StrictMode } from 'react'
2  import { createRoot } from 'react-dom/client'
3  import App from './App.jsx'
…
7    <StrictMode>
8      <App />
9    </StrictMode>,
```

Imagen 3. Extracto literal de src/main.jsx con el montaje de React.

Ejercicio 4. JSX y expresiones

Construí la bienvenida con JSX, texto reutilizable desde nombreAplicacion y el año calculado en ejecución. El CSS da formato al panel y el valor personalizado --panel-accent se aplica desde el componente.

JSX expressions and dynamic values

App.jsx · Extracto literal del archivo fuente

```
3
4  const nombreAplicacion = 'Clínica VetCare'
5
…
154            <div className="welcome-copy">
155              <p className="eyebrow">PANEL DE CONTROL <span>·</span> {new Date().getFullYear()}</p>
156              <h1 id="welcome-title">Un buen día para<br /><span>cuidar mejor.</span></h1>
```

Imagen 4. Extracto literal de src/App.jsx con expresiones JSX.

Ejercicio 5. Listas, map y key

Definí cinco pacientes y cuatro medicamentos con identificadores y campos compatibles con los serializers. Rendericé las tarjetas recorriendo cada arreglo con map(), usé id como key y formateé los pesos decimales con Number(...).toFixed(2).

Data arrays, map, stable keys, decimal formatting

App.jsx · Extracto literal del archivo fuente

```
  14
  15  const pacientes = [
  16    { id: 1, propietario: 1, nombre: 'Max', especie: 'Perro', raza: 'Mestizo', edad: 4, peso: '18.50' },
…
  22
  23  const medicamentos = [
  24    { id: 1, nombre: 'Antipulgas Canino', descripcion: 'Protección mensual para perros', stock: 12, precio:
  24      '38.50' },
…
 197            <div className="patient-grid">
 198              {pacientesVisibles.length > 0 ? pacientesVisibles.map((paciente) => {
 199                const propietario = propietariosPorId[paciente.propietario]
…
 203                    <div className="patient-info"><div className="patient-title"><h3>{paciente.nombre}</h3><span
 203      className="patient-id">#{String(paciente.id).padStart(3, '0')}</span></div><p>{paciente.raza} ·
 203      {paciente.edad} años</p><span className="owner-name">Tutor: {propietario.nombres}
 203      {propietario.apellidos}</span></div>
 204                    <div className="patient-weight"><strong>{Number(paciente.peso).toFixed(2)}
 204      <small>kg</small></strong><span>Peso actual</span></div>
 205                </article>
```

Imagen 5. Extracto literal de src/App.jsx con datos, map, key y formato decimal.

Ejercicio 6. Renderizado condicional

La búsqueda por nombre y el filtro por especie actualizan la lista. Si no hay coincidencias, se muestra “No hay registros” con una acción para limpiar el filtro; los medicamentos indican stock agotado y advierten cuando quedan pocas unidades.

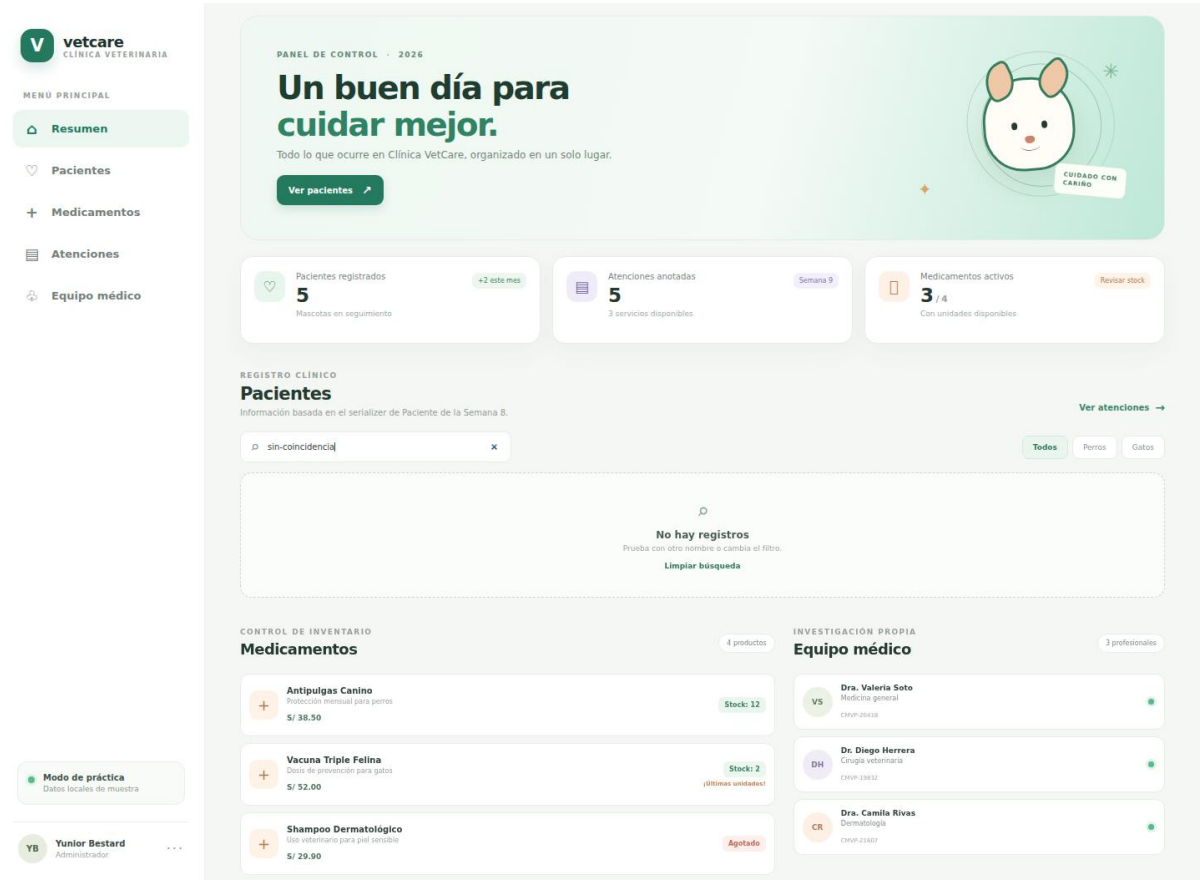


Imagen 6. Captura real del navegador con el estado vacío después de una búsqueda sin coincidencias.

Ejercicio 7. Relación 1:N con listas anidadas

Agrupé las atenciones por paciente y mostré el servicio relacionado, su fecha, estado y monto. Así se representa en la interfaz la relación Paciente–PacienteServicio–Servicio con una lista anidada.

Patient to service relationship data

App.jsx · Extracto literal del archivo fuente

```
42  // Forma de PacienteServicioSerializer: paciente y servicio vienen anidados desde la API.
43  const atenciones = [
44    {
…
46      paciente: { id: 1, nombre: 'Max', especie: 'Perro', raza: 'Mestizo', peso: '18.50', propietario: { id: 1,
46      nombres: 'Ana', apellidos: 'Rojas' } },
47      servicio: { id: 1, nombre: 'Consulta general', duracion_minutos: 30, precio: '45.00' },
48      cantidad: 1, monto: '45.00', estado: 'PAGADO', fecha_realizacion: '2026-10-16',
…
76  const pacientePorId = Object.fromEntries(pacientes.map((paciente) => [paciente.id, paciente]))
77  const atencionesPorPaciente = pacientes.map((paciente) => ({
78    ...paciente,
```

Imagen 7. Extracto literal de src/App.jsx con registros anidados de atención y servicio.

Ejercicio 8. Estilos y flujo de desarrollo

Apliqué hojas CSS responsivas, clases mediante className y un estilo inline para el acento visual. Verifiqué el proyecto con npm run lint y npm run build; la compilación Vite finalizó correctamente.

Responsive styles and dynamic accent

App.jsx · Extracto literal del archivo fuente

```
152
153         <section className="welcome-panel" aria-labelledby="welcome-title" style={{ '--panel-accent': '#bde8d7'
153      }}>
154           <div className="welcome-copy">
```

Imagen 8. Extracto literal de src/App.jsx con estilo dinámico; los estilos completos están en src/App.css.

Ejercicio 9. Entidades de la investigación

La pantalla incorpora Medicamento y Veterinario, junto con la atención intermedia PacienteServicio. Los nombres de campos siguen la API del curso, incluidos precio y monto como valores decimales serializados.

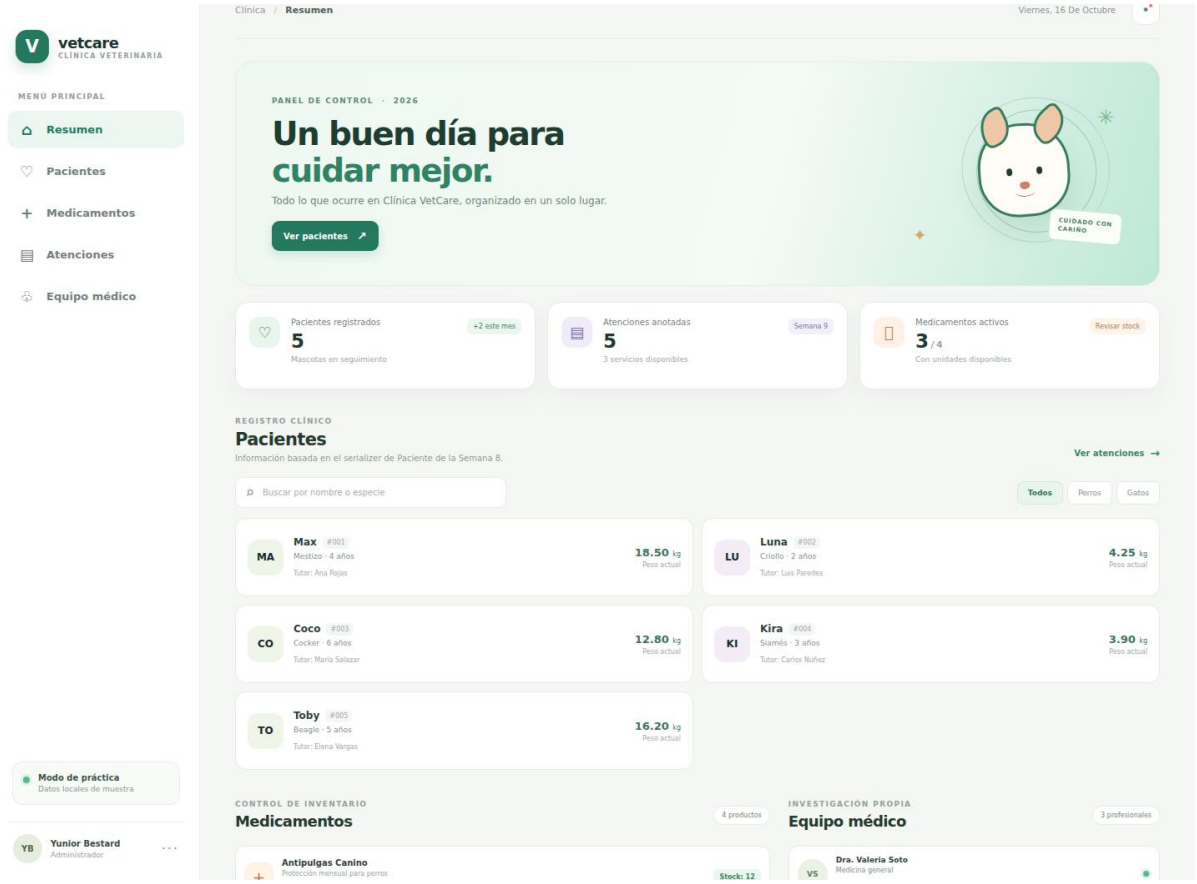


Imagen 9. Captura real del navegador con inventario y equipo médico.

Ejercicio 10. JSX y listas de las entidades

Rendericé las listas de Medicamento y Veterinario con map() y claves id. Cada tarjeta presenta la información principal del registro y mantiene estable la identidad de sus elementos.

Two entity lists with React map and key

App.jsx · Extracto literal del archivo fuente

```
214              <div className="medication-list">
215                {medicamentos.map((medicamento) => (
216                  <article className="medication-row" key={medicamento.id}>
217                    <div className="medication-symbol" aria-hidden="true">+</div>
…
230              <div className="team-list">
231                {veterinarios.map((veterinario) => (
232                  <article className="team-member" key={veterinario.id}>
233                    <div className="doctor-avatar" aria-hidden="true">{veterinario.nombre_completo.split('
233      ').slice(-2).map((parte) => parte[0]).join('')}</div>
```

Imagen 10. Extracto literal de src/App.jsx con las dos listas JSX.

Ejercicio 11. Condiciones y estilos de las entidades

El inventario diferencia medicamentos agotados, disponibles y con pocas unidades. Apliqué estilos de estado para que la condición se entienda visualmente, manteniendo el contenido de ejemplo en un arreglo independiente.

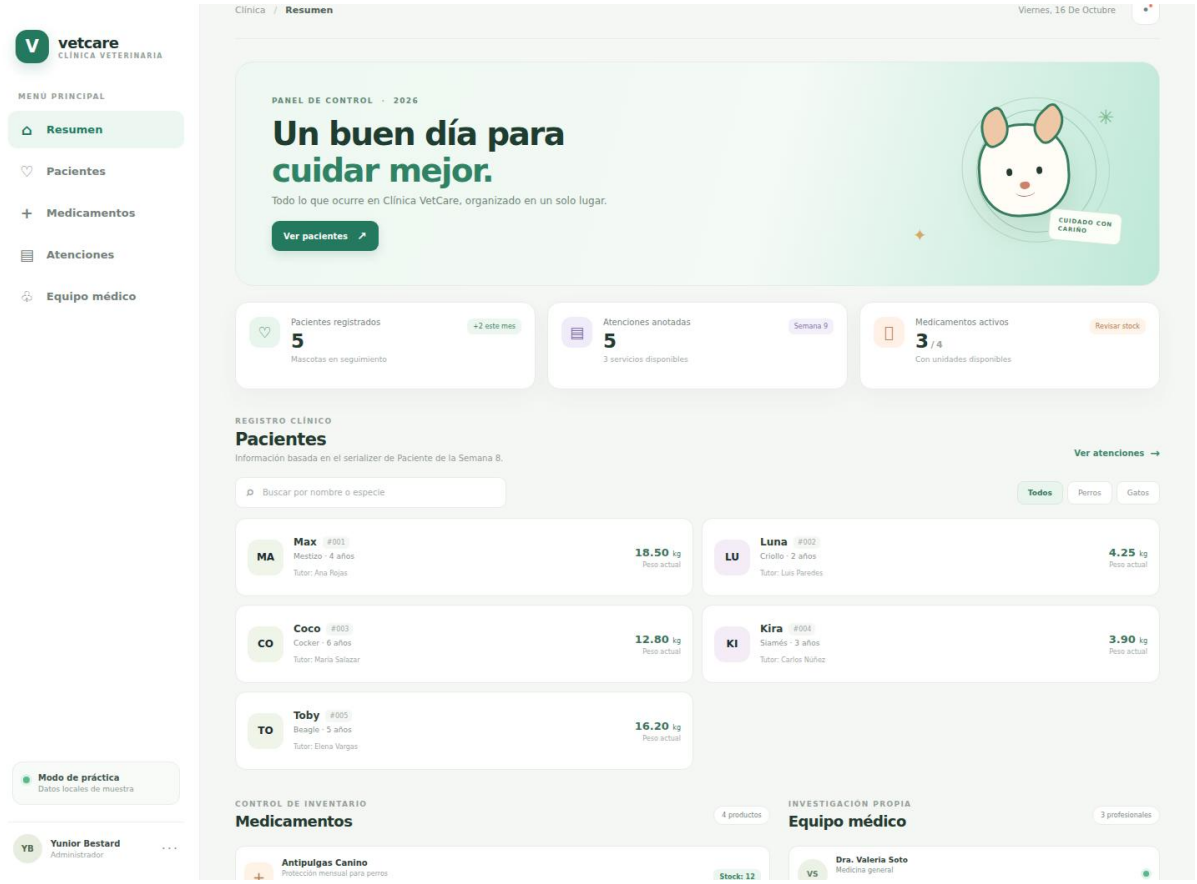


Imagen 11. Captura real del navegador que evidencia los estados de stock.

Ejercicio 12. Relación anidada con total por paciente

Rendericé las atenciones de cada paciente mediante una lista interna y presenté el total de atenciones. Cada fila muestra el servicio relacionado y datos de su registro intermedio.

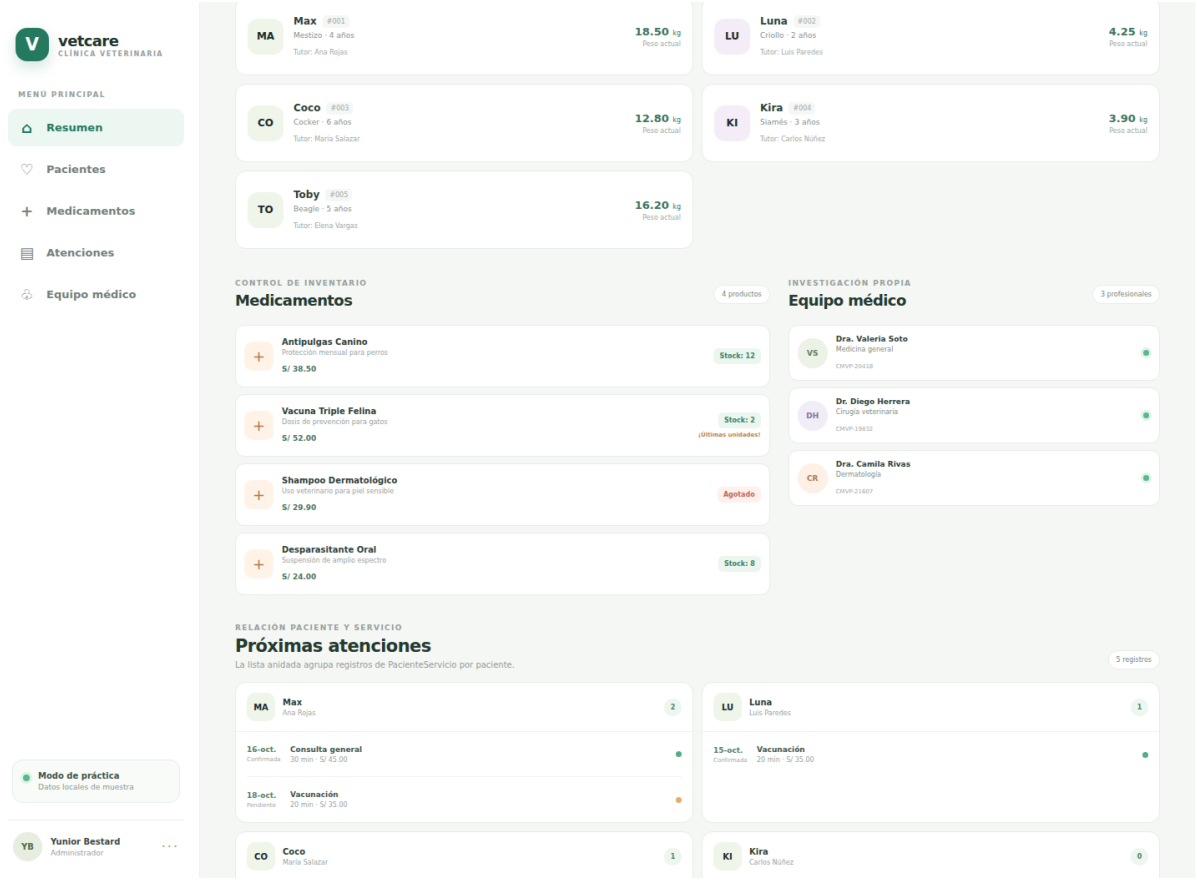


Imagen 12. Captura real del navegador con las atenciones y servicios agrupados.

Ejercicio 13. Verificación final

Probé la carga inicial, el estado vacío y el filtro de gatos en Chromium. La consola del navegador no registró errores ni advertencias. ESLint y la compilación de producción concluyeron con código de salida 0.

```
$ npm run lint
> eslint .
(sin errores; código de salida 0)

$ npm run build
✓ 30 modules transformed
✓ built successfully
```

La carpeta frontend/.gitignore excluye node_modules/ y dist/. La lista publicada contiene package.json, package-lock.json, configuración y código fuente, sin dependencias instaladas ni artefactos de compilación.

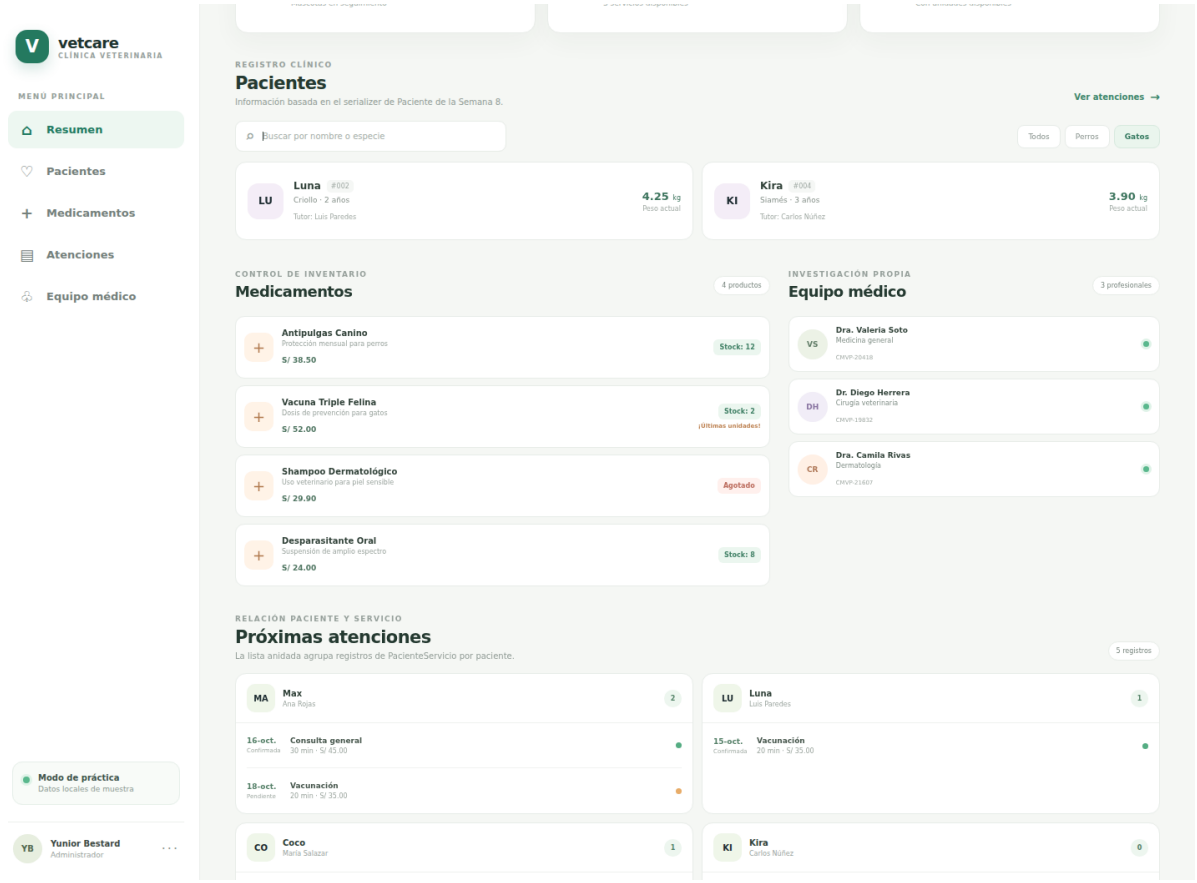


Imagen 13. Captura real del navegador tras filtrar pacientes de especie Gato; se muestran dos resultados.

Ejercicio 14. README y publicación en GitHub

Actualicé el README general respetando su esquema de Problemática, Requisitos Funcionales y Diseño del Modelo y Aplicación Creada, y agregué el README de Semana 9 con instrucciones de ejecución. Publiqué el proyecto en la carpeta Semana_09 del repositorio del curso.

Semana 9 README documentation

README.md · Extracto literal del archivo fuente

```
  2
  3  ## Problemática
  4
…
  6
  7  ## Requisitos funcionales
  8
…
 48  ```bash
 49  npm run lint
 50  npm run build
```

Imagen 14. Extracto del README de Semana 9 que documenta el alcance y los comandos de ejecución.

Repositorio: https://github.com/JesusCastillo2026/Desarrollo-de-Aplicaciones-Empresariales

Commit publicado en main: 53b8c999402c1ca8e678c61e3b1808d0a733c6b3 — “Desarrollar laboratorio 09 con React y Vite”.

Conclusiones

Quedó implementada una interfaz React que presenta y filtra pacientes, muestra el estado del inventario y agrupa atenciones por paciente. Los datos de prueba mantienen una forma compatible con la API desarrollada previamente, y el proyecto conserva el backend Django de la Semana 8. La verificación de lint, compilación y comportamiento en navegador fue satisfactoria.